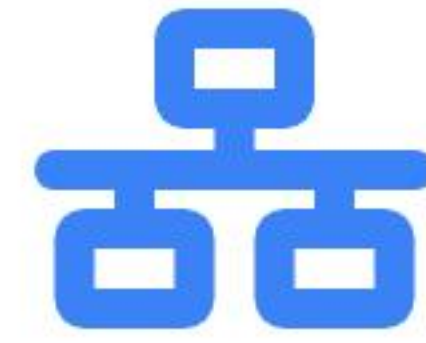




Estructuras de Datos

Jerárquicas

Fichas técnicas visuales de Árboles: análisis de complejidad, mecánicas de balanceo y aplicación en la industria del software.

Árbol Binario de Búsqueda (BST)

🔗 Identificación

Estructura jerárquica donde cada nodo tiene como máximo dos hijos. Valores menores se almacenan a la izquierda y mayores a la derecha.

🕒 Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

⚙️ Mecánicas

Mantiene el orden comparando valores durante el recorrido. Sin control automático, el árbol puede desbalancearse perdiendo su eficiencia técnica.

✅ Ideal Para

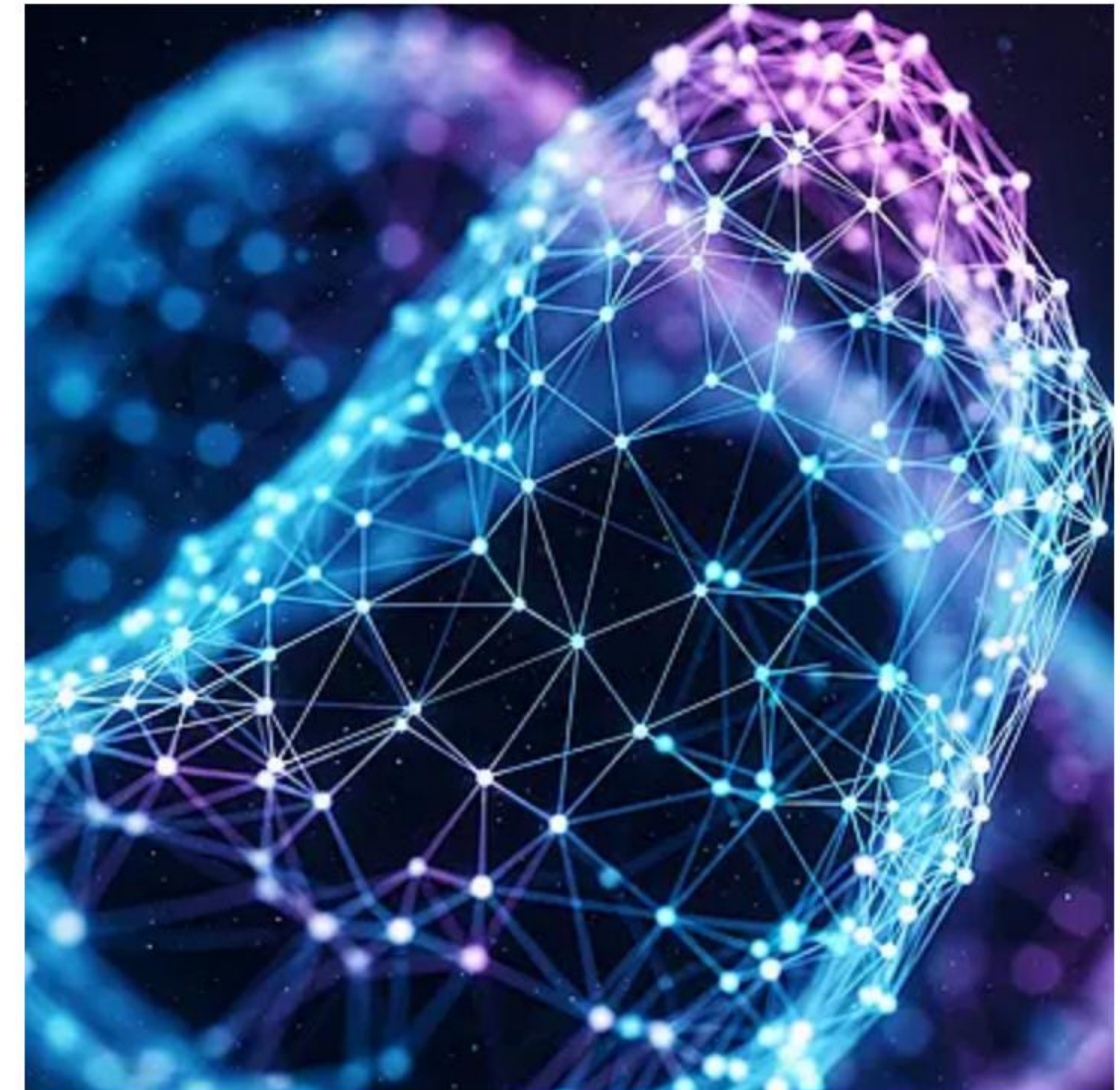
Búsquedas, inserciones y eliminaciones muy rápidas aprovechando su orden natural.

❌ Debilidad

Fácil desbalanceo. En el peor caso (datos insertados en orden), degenera a $O(n)$.

📁 Caso de Uso en la Industria

Sistemas de búsqueda en memoria y aplicaciones que requieren un almacenamiento de datos jerárquico básico y ordenado.



Árbol AVL (Auto-balanceado)

Identificación

Un Árbol Binario de Búsqueda con propiedad de auto-balanceo. Mantiene su altura equilibrada para asegurar rendimiento óptimo.

Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Mecánicas

Controla constantemente la diferencia de altura entre subárboles. Si un lado crece demasiado, efectúa rotaciones automáticas para recuperar el balance.

✓ Ideal Para

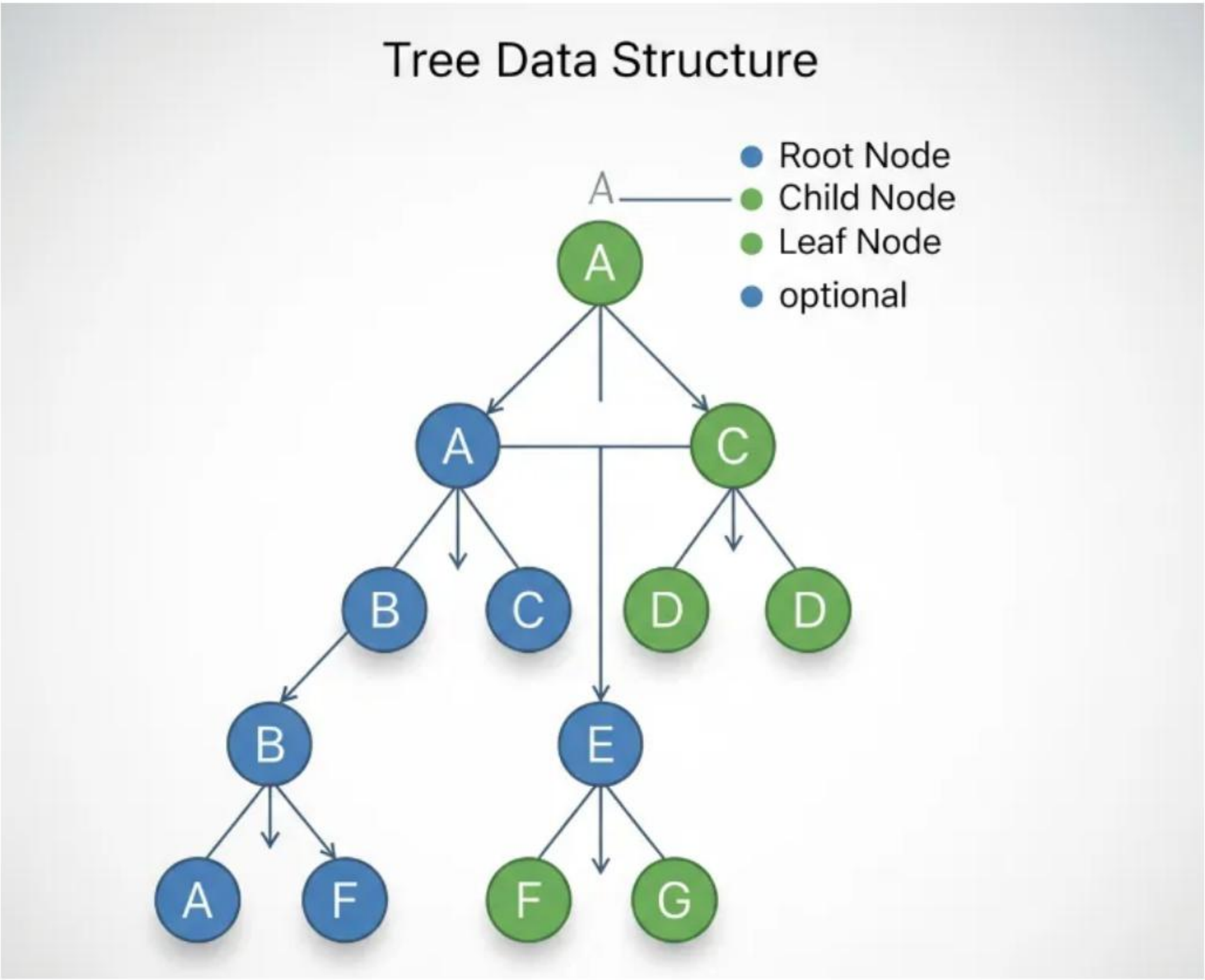
Búsquedas garantizadas a alta velocidad. Evita el desbalanceo extremo del BST.

✗ Debilidad

Implementación compleja. Las rotaciones añaden costo de procesamiento al escribir.

📁 Caso de Uso en la Industria

Sistemas de indexación y bases de datos en memoria donde las consultas de lectura son mucho más frecuentes que las escrituras.



Árbol B / B+

Identificación

Estructuras balanceadas multivariadas. Permiten que cada nodo almacene múltiples claves y referencias, creadas para gestionar datos masivos.

Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Mecánicas

Se auto-balancea combinando o dividiendo nodos llenos. En el Árbol B+, todos los datos reales están encapsulados en nodos hoja para facilitar su recorrido.

✓ Ideal Para

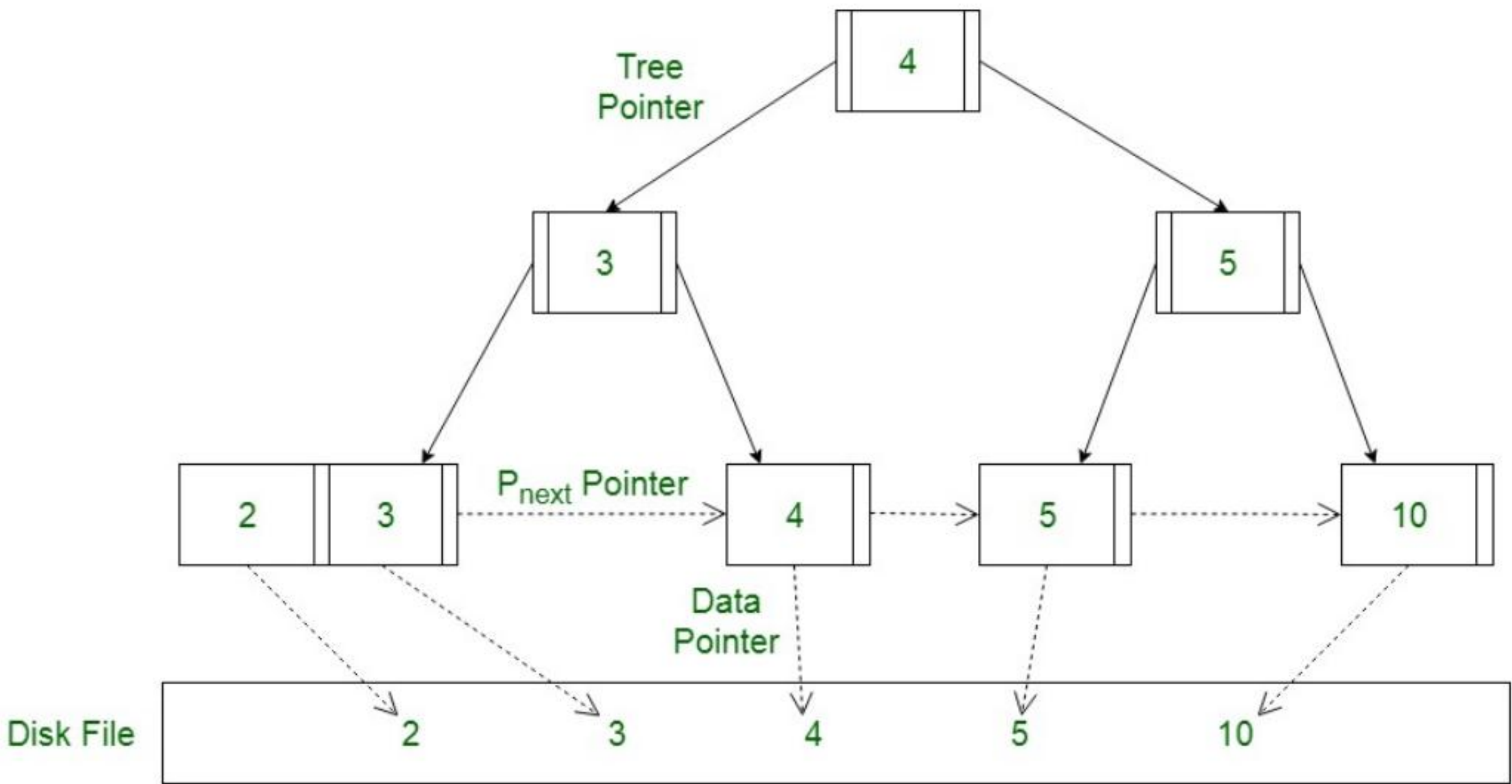
Reducir radicalmente la cantidad de lecturas al disco físico en operaciones masivas.

✗ Debilidad

Implementación sumamente intrincada y mayor consumo de memoria interna.

Caso de Uso en la Industria

Son el núcleo estructural de motores de bases de datos como MySQL,



Trie (Árbol de Prefijos)

A Identificación

Estructura especializada en procesar strings. Organiza diccionarios desglosando las palabras letra por letra a lo largo del árbol.

🕒 Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(m)$	$O(m)$
Inserción / Eli.	$O(m)$	$O(m)$

* m = longitud de la palabra

⚙ Mecánicas

Cada nodo almacena un carácter. Las palabras se revelan al recorrer desde la raíz a las hojas. Palabras similares comparten los mismos nodos de prefijo.

✅ Ideal Para

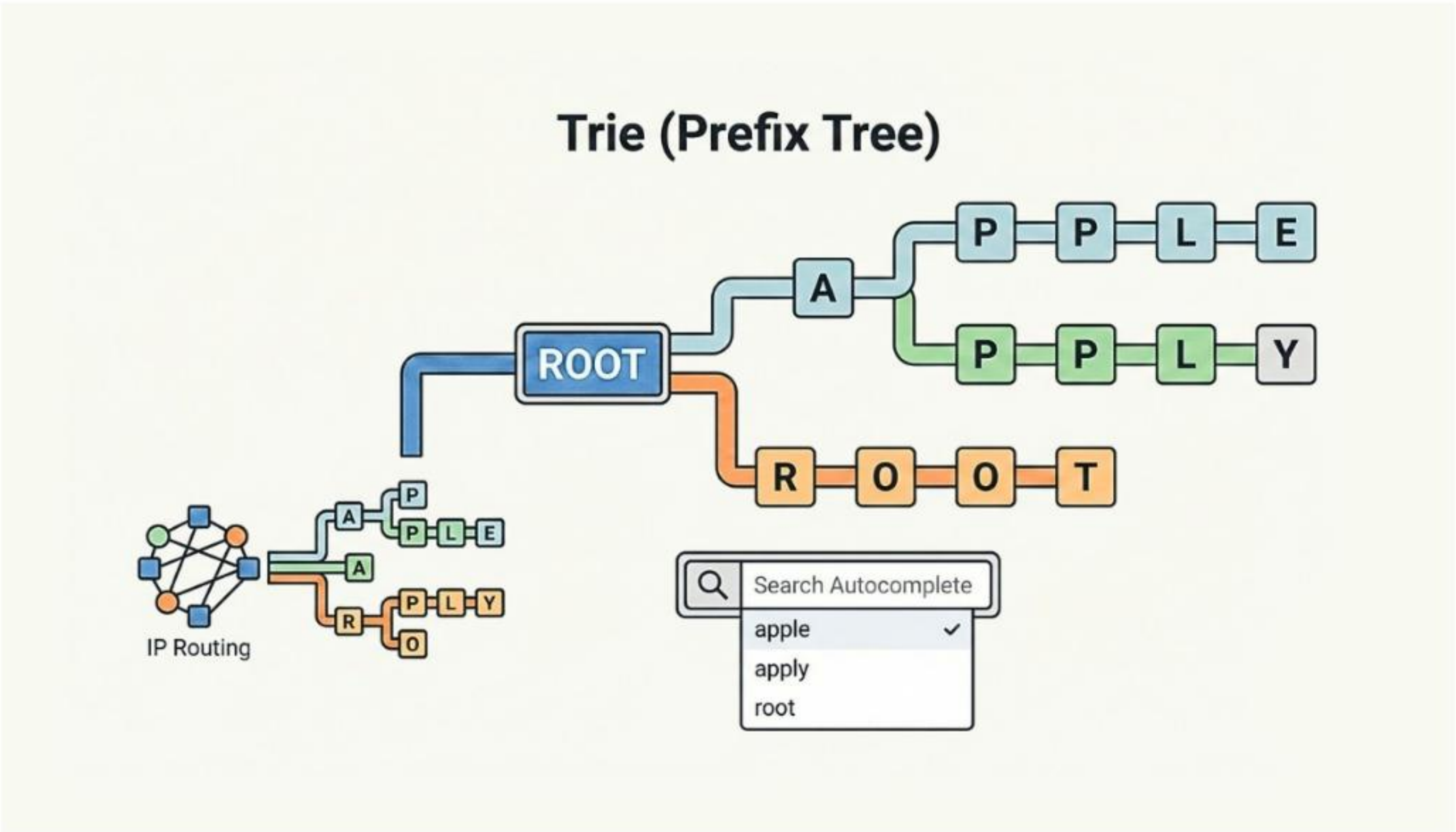
Búsqueda instantánea de texto
compartiendo prefijos eficientemente.

❌ Debilidad

Genera un consumo excesivo de
memoria para textos que no
comparten muchas letras.

📦 Caso de Uso en la Industria

Sistemas de autocompletado de texto, teclados predictivos, correctores ortográficos y enrutadores IP.



Heap (Montículo Min / Max)

Identificación

Estructura arbórea orientada al manejo de prioridades, enfocada en procesar rápidamente el valor máximo (Max Heap) o mínimo (Min Heap).

⚙️ Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda Gral.	$O(n)$	$O(n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

⚙️ Mecánicas

En un Max Heap, la raíz es siempre el valor mayor. Al insertar o extraer el nodo principal, el árbol se reorganiza solo para proteger la propiedad del Heap.

✅ Ideal Para

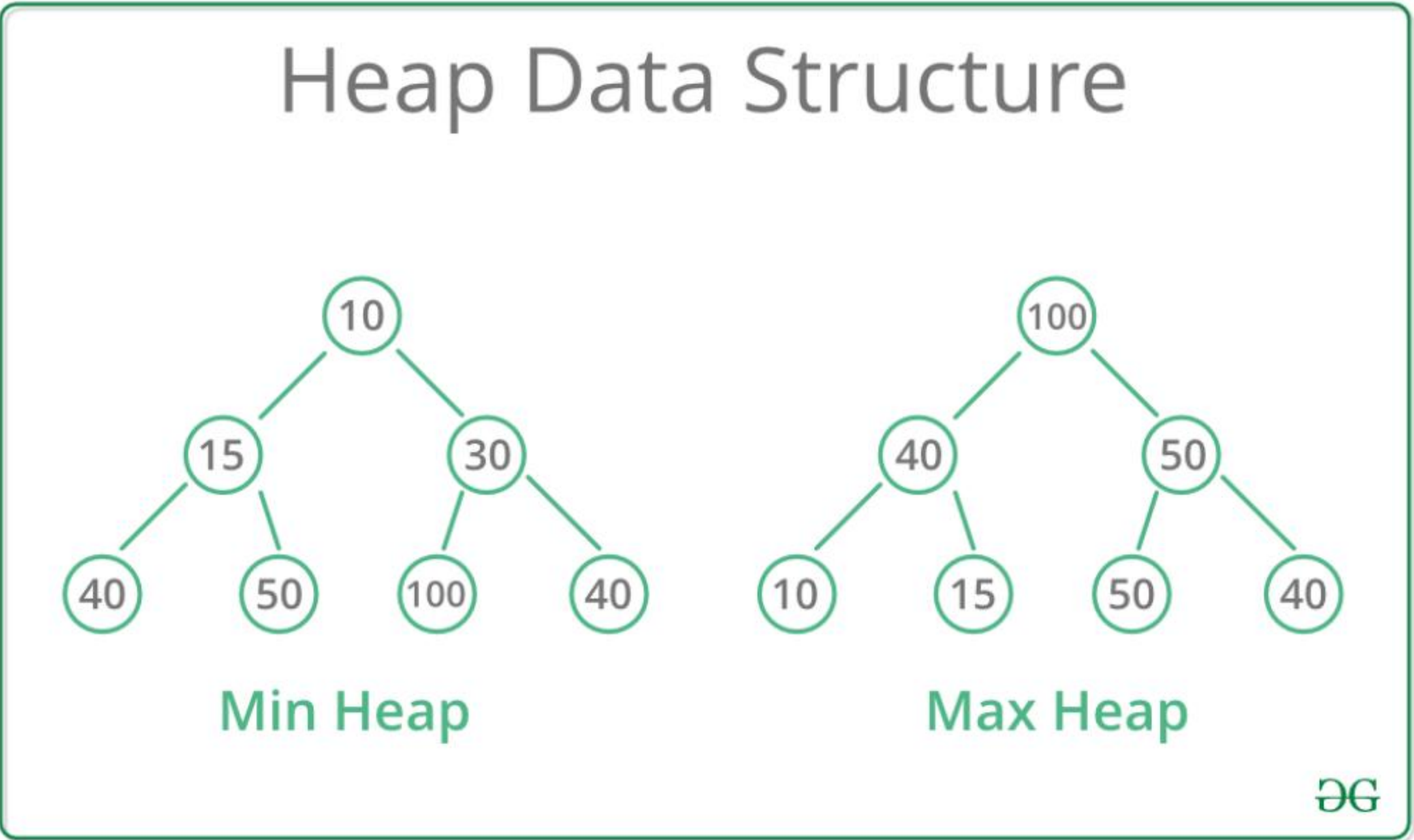
Acceso instantáneo $O(1)$ al elemento de mayor o menor prioridad.

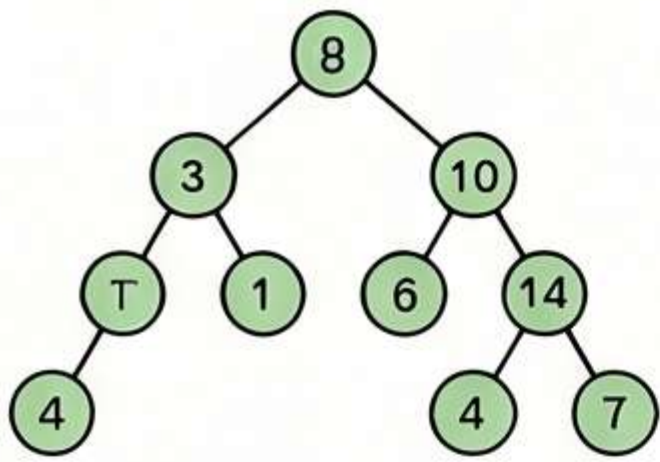
❌ Debilidad

Ineficiente para buscar elementos aleatorios o para mantener todo el conjunto ordenado.

📁 Caso de Uso en la Industria

Colas de prioridad, algoritmos de rutas (Dijkstra), algoritmos de ordenamiento





Árbol Binario de Búsqueda (BST)

Estructura Jerárquica de Dos Hijos: Es una estructura donde cada nodo tiene máximo dos hijos, organizando valores menores a la izquierda y mayores a la derecha.

Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda / Inserción / Eliminación	$O(\log n)$	$O(n)$

Mecánicas y Desafíos

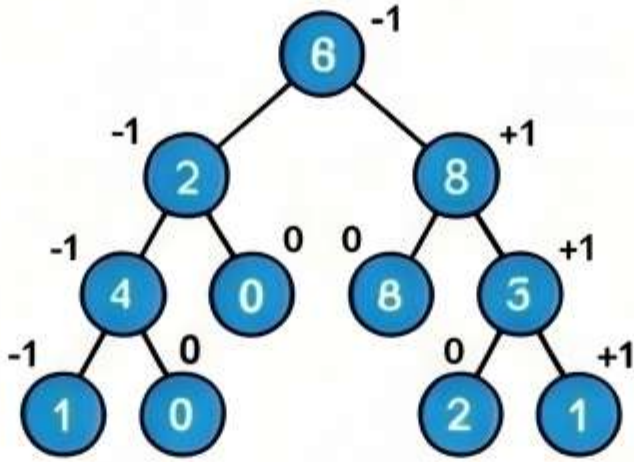


Eficiencia Variable: Aunque es rápido cuando está balanceado, puede degradarse y volverse feo como una lista enlazada si pierde su equilibrio.

Uso Industrial



Caso de Uso: Sistemas de Búsqueda Ideal para aplicaciones que requieren acceso rápido a información organizada de forma ascondante.



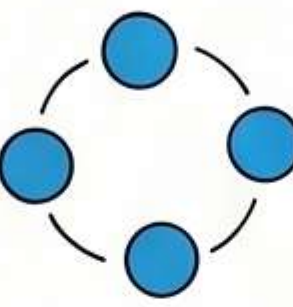
Árbol AVL

El Árbol que se Auto-Balancea: Un BST especializado que mantiene su altura equilibrada para garantizar operaciones rápidas en todo momento.

Análisis de Complejidad (Big O)

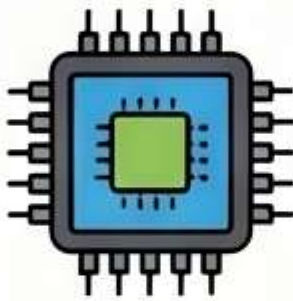
Operación	Caso Promedio	Peor Caso
Búsqueda / Inserción / Eliminación	$O(\log n)$	$O(\log n)$

Mecánicas y Desafíos

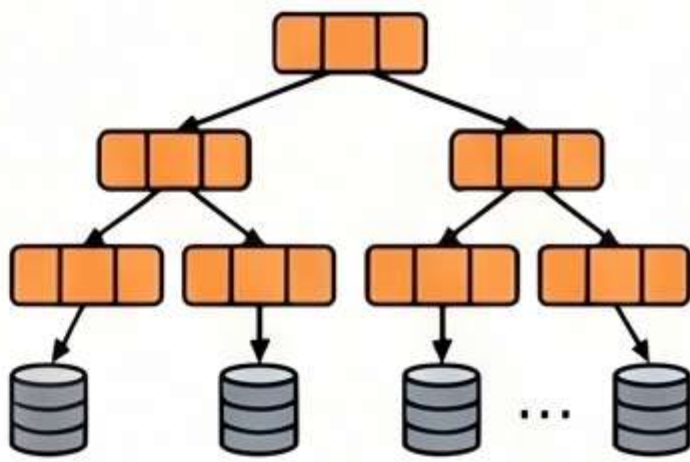


Rotaciones Automáticas: Si un subárbol crece demasiado, la estructura realiza movimientos internos (rotaciones) para recuperar el balance.

Uso Industrial



Caso de Uso: Bases de Datos en Memoria Utilizado en sistemas de indexación donde las búsquedas deben ser constantes y extremadamente veloces.



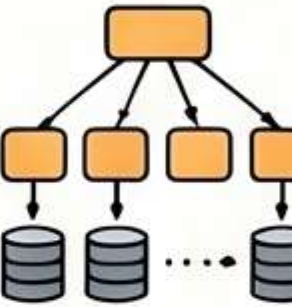
Árbol B / B+

Disaño para el Almacenamiento Masivo: Estructuras balanceadas que permiten múltiples hijos por nodo, reduciendo la necesidad de acceder frecuentemente al disco físico.

Análisis de Complejidad (Big O)

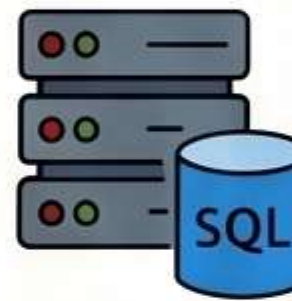
Operación	Caso Promedio	Peor Caso
Búsqueda / Inserción / Eliminación	$O(\log n)$	$O(\log n)$

Mecánicas y Desafíos

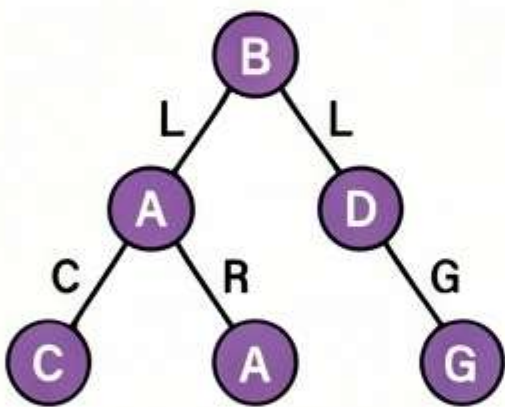


Optimización en Árbol B+: En la variante B+, todos los datos finales residen en las hojas, lo que facilita enormemente los recorridos secuenciales.

Uso Industrial



Caso de Uso: Motores de Bases de Datos Es la tecnología detrás de sistemas como PostgreSQL, MySQL y los sistemas de archivos modernos.



Trie (Árbol de Prefijos)

Especialista en Cadenas de Texto: Organiza palabras letra por letra, permitiendo que varios términos compartan el mismo camino si tienen prefijos comunes.

Análisis de Complejidad (Big O)

Búsqueda / Eliminación	$O(m)^*$	$O(m)^*$
------------------------	----------	----------

*m representa la longitud de la palabra.

Mecánicas y Desafíos

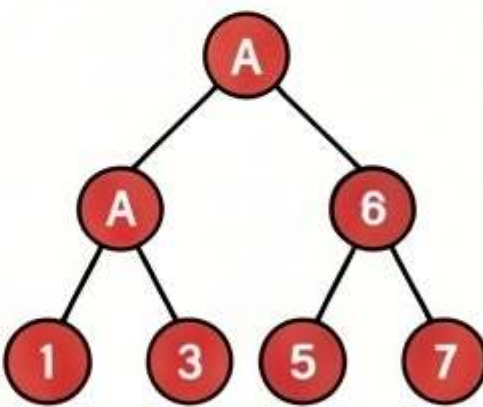


Rendimiento Basado en Longitud: Su velocidad no depende del número de palabras almacenadas, sino de la longitud de la palabra buscada (m).

Uso Industrial



Caso de Uso: Autocompletado y Teclados Es el motor detrás de los correctores ortográficos, teclados predictivos y buscadores de texto.



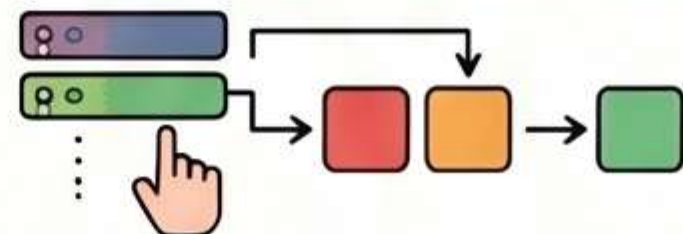
Heap (Montículo)

El Gestor de Prioridades: Una estructura arbórea diseñada para que el valor de mayor o menor importancia siempre está en la raíz (Max Heap o Min Heap).

Análisis de Complejidad (Big O)

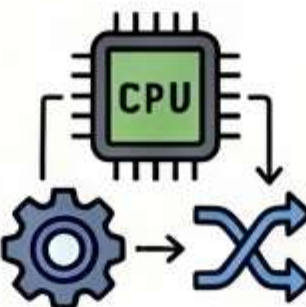
Inserción / Eliminación	$O(\log n)$	$O(\log n)$
Búsqueda	$O(n)$	$O(n)$

Mecánicas y Desafíos



Ineficiente para Búsquedas Generales: Aunque es excelente para manejar el elemento "superior", es lento para encontrar valores específicos en el resto del árbol.

Uso Industrial



Caso de Uso: Planificación de Procesos Utilizado por sistemas operativos para decidir qué tareas ejecutar a continuación y en algoritmos de rutas.